САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ
Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «Информатика»

Отчёт

По лабораторной работе №4

Вариант 119 (502511 % 132)

Выполнил:

Крамарь Михаил Романович

P3113

Проверила:

Авксентьева Елена Юрьевна, к.п.н., доцент

Санкт-Петербург 2025 г.

Содержание

Задание 3
Основные этапы вычисления 5
Обязательное задание.....5
Дополнительное задание №1.....9
Дополнительное задание №2.....13
Дополнительное задание №3.....15
Дополнительное задание №4.....20
Заключение.....25
Список использованных источников 26

Задание

1. Определить номер варианта как остаток деления на 132 своего идентификационного номера в ISU: например, 125598 / 132 = 16. В случае, если в обоих указанных днях недели нет занятий, то увеличить номер варианта на восемь. В случае, если занятий нет и в новом наборе дней, то продолжать увеличивать на восемь.
2. Изучить форму Бэкуса-Наура.
3. Изучить основные принципы организации формальных грамматик.
4. Изучить особенности языков разметки/форматов JSON, RON, HCL, YAML, TOML, INI, XML.
5. Понять устройство страницы с расписанием на примере расписания лектора:
https://itmo.ru/ru/schedule/3/125598/raspisanie_zanyatiy.htm
6. Исходя из структуры расписания конкретного дня, сформировать файл с расписанием в формате, указанном в задании в качестве исходного. При этом необходимо, чтобы хотя бы в одной из выбранных дней было не менее двух занятий (можно использовать своё персональное). В случае, если в данный день недели нет таких занятий, то увеличить номер варианта ещё на восемь.
7. Обязательное задание (позволяет набрать до 50 процентов от максимального числа баллов БаРС за данную лабораторную).
Написать программу на языке Python 3.x или любом другом, которая:

- осуществляет парсинг и конвертацию исходного файла в бинарный объект (=десериализацию);
- для решения задачи использует формальные грамматики; то есть ваш код должен уметь осуществлять парсинг и конвертацию любых данных, представленных в исходном формате, в данные, представленные в результирующем формате (как с готовыми библиотеками из дополнительного задания №2);
- не использовует готовые библиотеки, в том числе регулярные выражения в Python и библиотеки для загрузки файлов.

8. Дополнительное задание №1 (позволяет набрать +15 процентов от максимального числа баллов БаРС за данную лабораторную).
Написать программу на языке Python 3.x или любом другом, которая:

- осуществляет парсинг и конвертацию бинароного объекта, полученного в обязательном задании, в новый формат (=сериализацию);
- для решения задачи использует формальные грамматики;
- не использовует готовые библиотеки, в том числе регулярные выражения в Python и библиотеки для загрузки файлов.

9. Дополнительное задание №2 (позволяет набрать +10 процентов от максимального числа баллов БаРС за данную лабораторную).
а) Найти готовые библиотеки, осуществляющие аналогичный парсинг и конвертацию файлов (десериализацию и сериализацию).

b) Переписать исходный код и код из дополнительного задания №1, применив найденные библиотеки. Регулярные выражения также нельзя использовать.
c) Сравнить полученные результаты и объяснить их сходство/различие. Объяснение должно быть отражено в отчёте.

10. Дополнительное задание №3 (позволяет набрать +20 процентов от максимального числа баллов БаРС за данную лабораторную).
Переписать код из дополнительного задания №1, чтобы серализация происводилась в XML файл.

11. Дополнительное задание № 4 (позволяет набрать +5 процентов от максимального числа баллов БаРС за данную лабораторную).
a) Используя свою исходную программу из обязательного задания и программы из дополнительных заданий, сравнить стократное время выполнения парсинга + конвертации в цикле.
b) Проанализировать полученные результаты и объяснить их сходство/различие. Объяснение должно быть отражено в отчёте.

12.Проверить, что все пункты задания выполнены и выполнены верно.
13.Написать отчёт о проделанной работе.
14.Подготовиться к устным вопросам на защите.

119	YAML	JSON	Понедельник, суббота

Составленный YAML-файл с расписанием

```
schedule:
  selected_days:
    - day: "Среда"
      lessons:
        - time: "11:30-13:00"
          week: "2, 4, 6, 8, 10, 12, 14, 16"
          group: "ИНФОРМ КСиТ 2"
          audiency: "1216/0 (УСЛ) АУД."
          building: "ул.Ломоносова, д.9, лит.М"
          subject: "ИНФОРМАТИКА (ЛЕК) : АКТОВЫЙ ЗАЛ"
          format: "Очно-дистанционный"
        - time: "11:30-13:00"
          week: "2, 4, 6, 8, 10, 12, 14, 16"
          group: "ИНФОРМ СиППО 1"
          audiency: "1216/0 (УСЛ) АУД."
          building: "ул.Ломоносова, д.9, лит.М"
          subject: "ИНФОРМАТИКА (ЛЕК) : АКТОВЫЙ ЗАЛ"
          format: "Очно-дистанционный"
    - day: "Четверг"
      lessons:
        - time: "17:10-18:40"
          week: "2, 4, 6, 8, 10, 12, 14, 16"
          group: "ИНФОРМ (Р3116) 1.11"
          audiency: "3315 АУД."
          building: "ул.Ломоносова, д.9, лит.Е"
          subject: "ИНФОРМАТИКА (ЛАБ)"
          format: "Очно-дистанционный"
        - time: "18:50-20:20"
          week: "2, 4, 6, 8, 10, 12, 14, 16"
          group: "ИНФОРМ (Р3116) 1.11"
          audiency: "3315 АУД."
          building: "ул.Ломоносова, д.9, лит.Е"
          subject: "ИНФОРМАТИКА (ЛАБ)"
          format: "Очно-дистанционный"
    - day: "Суббота"
      lessons:
        - time: "8:10-9:40"
          week: "6, 7, 10, 11, 12, 13, 14, 15"
          group: "PRLCOMP 1"
          audiency: "1333 АУД."
          building: "Кронверкский пр., д.49, лит.А"
          subject: "PARALLEL COMPUTING / ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ (ЛЕК)"
          format: "Очно-дистанционный"
        - time: "9:50-11:20"
          week: "6, 7, 10, 11, 12, 13, 14, 15"
          group: "PRLCOMP 1.1"
          audiency: "1333 АУД."
```

```
      building: "Кронверкский пр., д.49, лит.А"
      subject: "PARALLEL COMPUTING / ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ (ЛАБ)"
      format: "Очно-дистанционный"
    - time: "11:30-13:00"
      week: "2, 4, 6, 8, 10, 12, 14, 16"
      group: "ИНФОРМ (Р3106) 1.1"
      audiency: "1333 АУД."
      building: "Кронверкский пр., д.49, лит.А"
      subject: "ИНФОРМАТИКА (ЛАБ)"
      format: "Очно-дистанционный"
    - time: "11:30-13:00"
      week: "3, 5, 7, 9, 11, 13, 15, 17"
      group: "ИНФОРМ (Р3107) 1.2"
      audiency: "1333 АУД."
      building: "Кронверкский пр., д.49, лит.А"
      subject: "ИНФОРМАТИКА (ЛАБ)"
      format: "Очно-дистанционный"
    - time: "13:30-15:00"
      week: "2, 4, 6, 8, 10, 12, 14, 16"
      group: "ИНФОРМ (Р3106) 1.1"
      audiency: "1333 АУД."
      building: "Кронверкский пр., д.49, лит.А"
      subject: "ИНФОРМАТИКА (ЛАБ)"
      format: "Очно-дистанционный"
    - time: "13:30-15:00"
      week: "3, 5, 7, 9, 11, 13, 15, 17"
      group: "ИНФОРМ (Р3107) 1.2"
      audiency: "1333 АУД."
      building: "Кронверкский пр., д.49, лит.А"
      subject: "ИНФОРМАТИКА (ЛАБ)"
      format: «Очно-дистанционный"
```

YAML_TO_BIN.py

```
lines = open('SourceFile.yaml').read().splitlines()
lines = [line for line in lines if line.strip() != ""]
#подсчёт количества пробелов(отступов) в начале строки для определения
вложенности
#отступом в yaml считается 2 пробела
def IndentLevel(line):
    counter = 0
    while line[counter] == ' ':
        counter += 1
    return counter

#stack = [ ({}, -1) ]
#           ^
#current_container
def ParseYaml(lines):
    data = {}
    stack = [(data, -1)]
    #стэк пар вида контейнер и уровень отступа
```

```
    for line in lines:
        indent = IndentLevel(line)
        line = line.strip()
        #если строка начинается с -, то элемент некого списка
        if line.startswith('-'):
            elem = line[1:].strip()
            #избавляемся от -
            key, value = elem.split(':', 1)
            key, value = key.strip(), value.strip().strip("")
            item = {key : value} if value else {key: {}}
            # достаём текущий контейнер
            while stack and indent <= stack[-1][1]:
                stack.pop()

            current_container = stack[-1][0]

            #если текущий контейнер не список — превращаем в список
            if not isinstance(current_container, list):
                #узнаём последний ключ прошлого контейнера
                old_container = stack[-2][0]
                last_key = list(old_container.keys())[-1]

                if not isinstance(old_container[last_key], list):
                    old_container[last_key] = []
                old_container[last_key].append(item)
                stack.append((item, indent))

            else:
                current_container.append(item)
                stack.append((item, indent))

        #иначе это ключ, у которого значение - словарь, или продолжение элемента
списка
        elif ':' in line:
            key, value = line.split(':', 1)
            key, value = key.strip(), value.strip().strip("")

            # выходим на нужный уровень по отступу
            while stack and indent <= stack[-1][1]:
                stack.pop()

            current_container = stack[-1][0]

            if value == "":
                current_container[key] = {}
                stack.append((current_container[key], indent))
            else:
                current_container[key] = value

    return data
```

```
def SerializeToBin(item_to_serialize):
    result = b''

    #если словарь
    if isinstance(item_to_serialize, dict):
        result += b'D' #своя метка dict
        for key, value in item_to_serialize.items():
            key_b = str(key).encode('utf-8') #кодирует строку в байты, но так как это
человекочитаемая кодировка, мы все равно увидим буквы в выводе
            result += len(key_b).to_bytes(2, 'big') + key_b
            #рекурсивно делаем то же самое со значением
            result += SerializeToBin(value)

    #если список
    elif isinstance(item_to_serialize, list):
        result += b'L'
        result += len(item_to_serialize).to_bytes(2, 'big')
        for item in item_to_serialize:
            result += SerializeToBin(item)
    else:
        result += b'S'
        value = str(item_to_serialize).encode('utf-8')
        result += len(value).to_bytes(2, 'big') + value

    return result

if __name__ == '__main__':
    data = ParseYaml(lines)
    print(data)
    print(SerializeToBin(data))
```

Полученный JSON-файл:


```
{
   "schedule": {
      "selected_days": [
         {
            "day": "Среда",
            "lessons": [
               {
                  "time": "11:30-13:00",
                  "week": "2, 4, 6, 8, 10, 12, 14, 16",
                  "group": "ИНФОРМ КСиТ 2",
                  "audiency": "1216/0 (УСЛ) АУД.",
                  "building": "ул.Ломоносова, д.9, лит.М",
                  "subject": "ИНФОРМАТИКА (ЛЕК) : АКТОВЫЙ ЗАЛ",
                  "format": "Очно-дистанционный"
               },
               {
                  "time": "11:30-13:00",
                  "week": "2, 4, 6, 8, 10, 12, 14, 16",
                  "group": "ИНФОРМ СиППО 1",
                  "audiency": "1216/0 (УСЛ) АУД.",
                  "building": "ул.Ломоносова, д.9, лит.М",
                  "subject": "ИНФОРМАТИКА (ЛЕК) : АКТОВЫЙ ЗАЛ",
                  "format": "Очно-дистанционный"
               }
            ]
         },
         {
            "day": "Четверг",
            "lessons": [
               {
                  "time": "17:10-18:40",
                  "week": "2, 4, 6, 8, 10, 12, 14, 16",
                  "group": "ИНФОРМ (Р3116) 1.11",
                  "audiency": "3315 АУД.",
                  "building": "ул.Ломоносова, д.9, лит.Е",
                  "subject": "ИНФОРМАТИКА (ЛАБ)",
                  "format": "Очно-дистанционный"
               },
               {
                  "time": "18:50-20:20",
                  "week": "2, 4, 6, 8, 10, 12, 14, 16",
                  "group": "ИНФОРМ (Р3116) 1.11",
                  "audiency": "3315 АУД.",
                  "building": "ул.Ломоносова, д.9, лит.Е",
                  "subject": "ИНФОРМАТИКА (ЛАБ)",
                  "format": "Очно-дистанционный"
               }
            ]
```

```
        },
        {
            "day": "Суббота",
            "lessons": [
                {
                    "time": "8:10-9:40",
                    "week": "6, 7, 10, 11, 12, 13, 14, 15",
                    "group": "PRLCOMP 1",
                    "audiency": "1333 АУД.",
                    "building": "Кронверкский пр., д.49, лит.А",
                    "subject": "PARALLEL COMPUTING / ПАРАЛЛЕЛЬНЫЕ
ВЫЧИСЛЕНИЯ (ЛЕК)",
                    "format": "Очно-дистанционный"
                },
                {
                    "time": "9:50-11:20",
                    "week": "6, 7, 10, 11, 12, 13, 14, 15",
                    "group": "PRLCOMP 1.1",
                    "audiency": "1333 АУД.",
                    "building": "Кронверкский пр., д.49, лит.А",
                    "subject": "PARALLEL COMPUTING / ПАРАЛЛЕЛЬНЫЕ
ВЫЧИСЛЕНИЯ (ЛАБ)",
                    "format": "Очно-дистанционный"
                },
                {
                    "time": "11:30-13:00",
                    "week": "2, 4, 6, 8, 10, 12, 14, 16",
                    "group": "ИНФОРМ (Р3106) 1.1",
                    "audiency": "1333 АУД.",
                    "building": "Кронверкский пр., д.49, лит.А",
                    "subject": "ИНФОРМАТИКА (ЛАБ)",
                    "format": "Очно-дистанционный"
                },
                {
                    "time": "11:30-13:00",
                    "week": "3, 5, 7, 9, 11, 13, 15, 17",
                    "group": "ИНФОРМ (Р3107) 1.2",
                    "audiency": "1333 АУД.",
                    "building": "Кронверкский пр., д.49, лит.А",
                    "subject": "ИНФОРМАТИКА (ЛАБ)",
                    "format": "Очно-дистанционный"
                },
                {
                    "time": "13:30-15:00",
                    "week": "2, 4, 6, 8, 10, 12, 14, 16",
                    "group": "ИНФОРМ (Р3106) 1.1",
                    "audiency": "1333 АУД.",
                    "building": "Кронверкский пр., д.49, лит.А",
                    "subject": "ИНФОРМАТИКА (ЛАБ)",
                    "format": "Очно-дистанционный"
                },
```

```
            {
                "time": "13:30-15:00",
                "week": "3, 5, 7, 9, 11, 13, 15, 17",
                "group": "ИНФОРМ (Р3107) 1.2",
                "audiency": "1333 АУД.",
                "building": "Кронверкский пр., д.49, лит.А",
                "subject": "ИНФОРМАТИКА (ЛАБ)",
                "format": "Очно-дистанционный"
            }
        ]
    }
  ]
 }
}
```

Dop_Task1.py:

```
#десериализация бинарного объекта из обязательного задания в структуру python
with open('../Task1/Task1_Binary.bin', 'rb') as bin_file:
    binary_data = bin_file.read()
    #прочитали содержимое файла, превратив его в строку

#превращаем в понятную питону структуру
def DeserializeFromBin(binary_data):

    def deserialize(data, offset):
        if offset >= len(data):
            return None, offset

        type_byte = data[offset:offset + 1]
        offset += 1

        if type_byte == b'D':  # Словарь
            result = {}
            while offset < len(data) and data[offset:offset + 1] not in (b'D', b'L', b'S'):
                key_length = int.from_bytes(data[offset:offset + 2], 'big')
                offset += 2
                key = data[offset:offset + key_length].decode('utf-8')
                offset += key_length
                value, offset = deserialize(data, offset)
                result[key] = value
            return result, offset

        elif type_byte == b'L':  # Список
            length = int.from_bytes(data[offset:offset + 2], 'big')
            offset += 2
            result = []
            for _ in range(length):
                item, offset = deserialize(data, offset)
                result.append(item)
            return result, offset
```

```
        elif type_byte == b'S':  # Строка
            length = int.from_bytes(data[offset:offset + 2], 'big')
            offset += 2
            value = data[offset:offset + length].decode('utf-8')
            offset += length
            return value, offset

    result, none = deserialize(binary_data, 0)

    return result

#превращаем питоновскую структуру в json-файл
#должно получится что-то вроде такого:
#{
# "schedule": {
#   "sel_days": [
#     {
#       "day": "Среда",
#       "time": "12"
#     }
#   ]
# }
#}
def ToJson(data, indent = 0):
    space = "    " * indent

    if isinstance(data, dict):
        items = []
        for key, value in data.items():
            json_value = ToJson(value, indent + 1)
            items.append(f'{space}    "{key}": {json_value}')
        return '{\n' + ',\n'.join(items) + '\n' + space + '}'

    elif isinstance(data, list):
        items = []
        for item in data:
            items.append(f'{space}    {ToJson(item, indent + 1)}')
        return '[\n' + ',\n'.join(items) + '\n' + space + ']'

    elif isinstance(data, str):
        return f'"{data}"'

if __name__ == '__main__':
    data = DeserializeFromBin(binary_data)
    json_text = ToJson(data)
    with open("Dop_Task1.json", "w", encoding="utf-8") as f:
        f.write(json_text)
```

Дополнительное задание №2

Dop_Task2.py:

```
#часть 1
import yaml
import pickle

#выполним сериализацию из yaml в python структуру, а потом сериализуем в бинарный файл
with open('SourceFile.yaml') as file:
    data = yaml.load(file, Loader = yaml.SafeLoader)

with open('Dop_Task2_BinaryFile.bin', 'wb') as file_b:
    pickle.dump(data, file_b)
    #записываем в файл нашу data в байтовом виде

#часть 2
import json

with open('Dop_Task2_BinaryFile.bin', 'rb') as file_b:
    binary_data = file_b.read()
    #читаем байты, записанные в data

new_data = pickle.loads(binary_data)
#с помощью библиотеки десериализуем байты в структу питончика
with open('Dop_Task2_JSON.json', 'w') as file_js:
    json.dump(new_data, file_js, ensure_ascii=False, indent=4)
    #записываем python объект в файл в формате json
    #ensure_ascii=False - если ensure_ascii = True, все не-ASCII символы в выводе будут \uXXXX, и результатом будет строка, содержащая только ASCII символы. Если ensure_ascii = False, строки запишутся как есть.
    #indent=4 - просто уровень отступа
```

Выводы:

Если сравнивать мою собственную сериализацию/десериализацию с готовыми библиотечными решениями (json и yaml), то можно отметить следующие важные отличия:

В моей ручной реализации структура данных кодируется в бинарном формате с явным указанием типа, длины строк и рекурсивной обработкой вложенных структур. Возможно как раз рекурсивный подход и увеличивает время работы моей программы. Это обеспечивает компактное хранение, но делает формат немного трудным для прочтения и достаточно сложным в отладке…

Библиотеки json и yaml реализуют сериализацию на уровне языка Python, а значит автоматически обрабатывают списочки, словари, строки.

По результатам тестирований могу сделать вывод, что библиотечные методы работают немного быстрее и, конечно же, надёжнее. Результаты сериализации совпадают с данными, восстановленными вручную, что не может не радовать!!!

Дополнительное задание №3

Полученный XML-файл:

```
<?xml version='1.0' encoding='UTF-8'?>
<data>
  <schedule>
    <selected_days>
      <item>
        <day>Среда</day>
        <lessons>
          <item>
            <time>11:30-13:00</time>
            <week>2, 4, 6, 8, 10, 12, 14, 16</week>
            <group>ИНФОРМ КСиТ 2</group>
            <audiency>1216/0 (УСЛ) АУД.</audiency>
            <building>ул.Ломоносова, д.9, лит.М</building>
            <subject>ИНФОРМАТИКА (ЛЕК) : АКТОВЫЙ ЗАЛ</subject>
            <format>Очно-дистанционный</format>
          </item>
          <item>
            <time>11:30-13:00</time>
            <week>2, 4, 6, 8, 10, 12, 14, 16</week>
            <group>ИНФОРМ СиППО 1</group>
            <audiency>1216/0 (УСЛ) АУД.</audiency>
            <building>ул.Ломоносова, д.9, лит.М</building>
            <subject>ИНФОРМАТИКА (ЛЕК) : АКТОВЫЙ ЗАЛ</subject>
            <format>Очно-дистанционный</format>
          </item>
        </lessons>
      </item>
      <item>
        <day>Четверг</day>
        <lessons>
          <item>
            <time>17:10-18:40</time>
            <week>2, 4, 6, 8, 10, 12, 14, 16</week>
            <group>ИНФОРМ (Р3116) 1.11</group>
            <audiency>3315 АУД.</audiency>
            <building>ул.Ломоносова, д.9, лит.Е</building>
            <subject>ИНФОРМАТИКА (ЛАБ)</subject>
            <format>Очно-дистанционный</format>
          </item>
          <item>
            <time>18:50-20:20</time>
            <week>2, 4, 6, 8, 10, 12, 14, 16</week>
            <group>ИНФОРМ (Р3116) 1.11</group>
            <audiency>3315 АУД.</audiency>
            <building>ул.Ломоносова, д.9, лит.Е</building>
            <subject>ИНФОРМАТИКА (ЛАБ)</subject>
            <format>Очно-дистанционный</format>
          </item>
        </lessons>
```

```
    </item>
    <item>
      <day>Суббота</day>
      <lessons>
        <item>
          <time>8:10-9:40</time>
          <week>6, 7, 10, 11, 12, 13, 14, 15</week>
          <group>PRLCOMP 1</group>
          <audiency>1333 АУД.</audiency>
          <building>Кронверкский пр., д.49, лит.А</building>
          <subject>PARALLEL COMPUTING / ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ
(ЛЕК)</subject>
          <format>Очно-дистанционный</format>
        </item>
        <item>
          <time>9:50-11:20</time>
          <week>6, 7, 10, 11, 12, 13, 14, 15</week>
          <group>PRLCOMP 1.1</group>
          <audiency>1333 АУД.</audiency>
          <building>Кронверкский пр., д.49, лит.А</building>
          <subject>PARALLEL COMPUTING / ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ
(ЛАБ)</subject>
          <format>Очно-дистанционный</format>
        </item>
        <item>
          <time>11:30-13:00</time>
          <week>2, 4, 6, 8, 10, 12, 14, 16</week>
          <group>ИНФОРМ (Р3106) 1.1</group>
          <audiency>1333 АУД.</audiency>
          <building>Кронверкский пр., д.49, лит.А</building>
          <subject>ИНФОРМАТИКА (ЛАБ)</subject>
          <format>Очно-дистанционный</format>
        </item>
        <item>
          <time>11:30-13:00</time>
          <week>3, 5, 7, 9, 11, 13, 15, 17</week>
          <group>ИНФОРМ (Р3107) 1.2</group>
          <audiency>1333 АУД.</audiency>
          <building>Кронверкский пр., д.49, лит.А</building>
          <subject>ИНФОРМАТИКА (ЛАБ)</subject>
          <format>Очно-дистанционный</format>
        </item>
        <item>
          <time>13:30-15:00</time>
          <week>2, 4, 6, 8, 10, 12, 14, 16</week>
          <group>ИНФОРМ (Р3106) 1.1</group>
          <audiency>1333 АУД.</audiency>
          <building>Кронверкский пр., д.49, лит.А</building>
          <subject>ИНФОРМАТИКА (ЛАБ)</subject>
          <format>Очно-дистанционный</format>
        </item>
```

```
        <item>
          <time>13:30-15:00</time>
          <week>3, 5, 7, 9, 11, 13, 15, 17</week>
          <group>ИНФОРМ (Р3107) 1.2</group>
          <audiency>1333 АУД.</audiency>
          <building>Кронверкский пр., д.49, лит.А</building>
          <subject>ИНФОРМАТИКА (ЛАБ)</subject>
          <format>Очно-дистанционный</format>
        </item>
      </lessons>
    </item>
  </selected_days>
 </schedule>
</data>
```

Dop_Task3.py:

```
#десериализация бинарного объекта из обязательного задания в структуру python, а потом в xml-формат
with open('../Task1/Task1_Binary.bin', 'rb') as bin_file:
    data_in_file = bin_file.read()
    #прочитали содержимое файла, превратив его в строку

#превращаем в понятную питону структуру
def DeserializeFromBin(binary_data):
    def deserialize(data, offset):
        if offset >= len(data):
            return None, offset

        type_byte = data[offset:offset + 1]
        offset += 1

        if type_byte == b'D':  # Словарь
            result = {}
            while offset < len(data) and data[offset:offset + 1] not in (b'D', b'L', b'S'):
                key_length = int.from_bytes(data[offset:offset + 2], 'big')
                offset += 2
                key = data[offset:offset + key_length].decode('utf-8')
                offset += key_length
                value, offset = deserialize(data, offset)
                result[key] = value
            return result, offset

        elif type_byte == b'L':  # Список
            length = int.from_bytes(data[offset:offset + 2], 'big')
            offset += 2
            result = []
            for _ in range(length):
                item, offset = deserialize(data, offset)
                result.append(item)
            return result, offset
```

```
        elif type_byte == b'S':  # Строка
            length = int.from_bytes(data[offset:offset + 2], 'big')
            offset += 2
            value = data[offset:offset + length].decode('utf-8')
            offset += length
            return value, offset

    result, none = deserialize(binary_data, 0)

    return result

def ToXML(data, indent):
    result = ""
    space = "  " * indent
    #тут за отступ берём 2 пробела

    #если словарь
    if isinstance(data, dict):
        for key, value in data.items():
            if isinstance(value, (dict, list)):
                result += f"{space}<{key}>\n{ToXML(value, indent + 1)}{space}</{key}>\n"
            else:
                result += f"{space}<{key}>{value}</{key}>\n"
            #тут происходит форматирование в формат, как в xml:
            #<schedule>
            #  <selected_days>
            #    <item>
            #      <day>Среда</day>
            #       .....

    #если список
    elif isinstance(data, list):
        for item in data:
            if isinstance(item, (dict, list)):
                result += f"{space}<item>\n{ToXML(item, indent + 1)}{space}</item>\n"
                #тут по аналогии со случаем, когда у нас словарь
            else:
                result += f"{space}<item>{item}</item>\n"

    else:
        result += f"{space}{data}\n"

    return result

if __name__ == '__main__':
    data = DeserializeFromBin(data_in_file)

    xml_text = "<?xml version='1.0' encoding='UTF-8'?>\n<data>\n"
    xml_text += ToXML(data, indent=1)
    xml_text += "</data>"
```

```
with open("Dop_Task3.xml", "w", encoding="utf-8") as f:
    f.write(xml_text)
```

Дополнительное задание №4

Dop_Task4.py:

```
import time

#скопируем сюда все нужные нам функции из наших заданий
def IndentLevel(line):
    counter = 0
    while line[counter] == ' ':
        counter += 1
    return counter

def ParseYaml(lines):
    data = {}
    stack = [(data, -1)]
    for line in lines:
        indent = IndentLevel(line)
        line = line.strip()
        if line.startswith('-'):
            elem = line[1:].strip()
            key, value = elem.split(':', 1)
            key, value = key.strip(), value.strip().strip("'")
            item = {key : value} if value else {key: {}}
            while stack and indent <= stack[-1][1]:
                stack.pop()
            current_container = stack[-1][0]
            if not isinstance(current_container, list):
                old_container = stack[-2][0]
                last_key = list(old_container.keys())[-1]
                if not isinstance(old_container[last_key], list):
                    old_container[last_key] = []
                old_container[last_key].append(item)
                stack.append((item, indent))
            else:
                current_container.append(item)
                stack.append((item, indent))
        elif ':' in line:
            key, value = line.split(':', 1)
            key, value = key.strip(), value.strip().strip("'")
            while stack and indent <= stack[-1][1]:
                stack.pop()
            current_container = stack[-1][0]
            if value == '':
                current_container[key] = {}
                stack.append((current_container[key], indent))
            else:
                current_container[key] = value
    return data

def SerializeToBin(item_to_serialize):
    result = b''
    if isinstance(item_to_serialize, dict):
```

```
        result += b'D'
        for key, value in item_to_serialize.items():
            key_b = str(key).encode('utf-8')
            result += len(key_b).to_bytes(2, 'big') + key_b
            result += SerializeToBin(value)
    elif isinstance(item_to_serialize, list):
        result += b'L'
        result += len(item_to_serialize).to_bytes(2, 'big')
        for item in item_to_serialize:
            result += SerializeToBin(item)
    else:
        result += b'S'
        value = str(item_to_serialize).encode('utf-8')
        result += len(value).to_bytes(2, 'big') + value
    return result

# Из Dop_Task1.py
def DeserializeFromBin(binary_data):
    def deserialize(data, offset):
        if offset >= len(data):
            return None, offset
        type_byte = data[offset:offset + 1]
        offset += 1
        if type_byte == b'D':
            result = {}
            while offset < len(data) and data[offset:offset + 1] not in (b'D', b'L', b'S'):
                key_length = int.from_bytes(data[offset:offset + 2], 'big')
                offset += 2
                key = data[offset:offset + key_length].decode('utf-8')
                offset += key_length
                value, offset = deserialize(data, offset)
                result[key] = value
            return result, offset
        elif type_byte == b'L':
            length = int.from_bytes(data[offset:offset + 2], 'big')
            offset += 2
            result = []
            for _ in range(length):
                item, offset = deserialize(data, offset)
                result.append(item)
            return result, offset
        elif type_byte == b'S':
            length = int.from_bytes(data[offset:offset + 2], 'big')
            offset += 2
            value = data[offset:offset + length].decode('utf-8')
            offset += length
            return value, offset
    result, _ = deserialize(binary_data, 0)
    return result

def ToJson(data, indent = 0):
```

```
    space = "    " * indent
    if isinstance(data, dict):
        items = []
        for key, value in data.items():
            json_value = ToJson(value, indent + 1)
            items.append(f'{space}    "{key}": {json_value}')
        return '{\n' + ',\n'.join(items) + '\n' + space + '}'
    elif isinstance(data, list):
        items = []
        for item in data:
            items.append(f'{space}    {ToJson(item, indent + 1)}')
        return '[\n' + ',\n'.join(items) + '\n' + space + ']'
    elif isinstance(data, str):
        return f'"{data}"'

# Из Dop_Task3.py
def ToXML(data, indent):
    result = ""
    space = "  " * indent
    if isinstance(data, dict):
        for key, value in data.items():
            if isinstance(value, (dict, list)):
                result += f"{space}<{key}>\n{ToXML(value, indent + 1)}{space}</{key}>\n"
            else:
                result += f"{space}<{key}>{value}</{key}>\n"
    elif isinstance(data, list):
        for item in data:
            if isinstance(item, (dict, list)):
                result += f"{space}<item>\n{ToXML(item, indent + 1)}{space}</item>\n"
            else:
                result += f"{space}<item>{item}</item>\n"
    else:
        result += f"{space}{data}\n"
    return result


#считываем содержимое бинарного файла
with open('Task1/Task1_Binary.bin') as bin_file:
    binary_data = bin_file.read()

#тест обязательного задания Task1
lines = open('SourceFile.yaml').read().splitlines()
lines = [line for line in lines if line.strip() != ""]

start = time.time()
for i in range(100):
    data = ParseYaml(lines)
    bin_data = SerializeToBin(data)
time_task1 = time.time() - start
start = time.time()
```

```
#теперь протестируем задание Dop_Task1.py
start = time.time()
for i in range(100):
    data = DeserializeFromBin(bin_data)
    json_text = ToJson(data)
time_dop1 = time.time() - start

#теперь протестим задание Dop_Task2.py
import yaml
import pickle
import json
with open('SourceFile.yaml') as file:
    yaml_data = yaml.load(file, Loader = yaml.SafeLoader)

start = time.time()
for i in range(100):
    pickle_data = pickle.dumps(yaml_data)
    new_data = pickle.loads(pickle_data)
    json.dumps(new_data, ensure_ascii=False, indent=4)
time_dop2 = time.time() - start

#самое время протестировать задание Dop_Task3.py
start = time.time()
for i in range(100):
    data = DeserializeFromBin(bin_data)
    xml_text = "<?xml version='1.0' encoding='UTF-8'?>\n<data>\n"
    xml_text += ToXML(data, indent = 1)
    xml_text += "</data>"
time_dop3 = time.time() - start

#РЕЗУЛЬТАТЫ
print('РЕЗУЛЬТАТЫ 100 ЗАПУСКОВ:')
print(f'Task1(YAML_TO_BIN).py, результат: {time_task1:.3f} сек')
print(f'Dop_Task1.py, результат: {time_dop1:.3f} сек')
print(f'Dop_Task2.py, результат: {time_dop2:.3f} сек')
print(f'Dop_Task3.py, результат: {time_dop3:.3f} сек')
```

Результаты работы программы:

РЕЗУЛЬТАТЫ 100 ЗАПУСКОВ:
Task1(YAML_TO_BIN).py, результат: 0.011 сек
Dop_Task1.py, результат: 0.009 сек
Dop_Task2.py, результат: 0.007 сек
Dop_Task3.py, результат: 0.008 сек

Выводы:

Библиотечные функции (yaml, pickle, json) написаны на языке C и поэтому сильно оптимизированы

Парсеры, которые пишу я на Python медленнее из-за их интерпретируемой природы, всё-таки это интерпретируемый язык как-никак

XML-генерация требует больше строковых операций и форматирования из-за того, как собственно XML-файл устроен и выглядит

Из-за рекурсивных алгоритмов парсеры, написанные вручную, очевидно, работают медленнее, чем уже реализованные библиотечные аналоги

Заключение

Во время выполнения лабораторной работы №4 я узнал о таких языках разметки, как YAML и JSON, научился с ними работать и переводить один в другой при помощи средств языка Python. Поработал с сериализацией в бинарный объект, узнал об его представлении. Также научился работать с некоторыми библиотеками для парсинга, такими как json и yaml.

Список использованных источников

1. https://books.ifmo.ru/file/pdf/2256.pdf
2. https://ru.wikipedia.org/wiki/Формальная_грамматика
3. https://habr.com/ru/post/309242/
4. https://ru.wikipedia.org/wiki/Форма_Бэкуса _—_Наура